



抖音网络优化实践

谈晶晶

February, 2023

导师自我介绍



谈晶晶

毕业于南京大学，先后就职于 Marvell、Dynosense、爱奇艺等公司，并于2022年加入字节跳动抖音 Android 客户端基础技术团队，目前专注于抖音 Android 的网络优化工作，对 Android 网络优化有较为丰富的经验。



01 背景介绍

02 网络监控

03 体验优化

04 带宽优化

05 长期规划

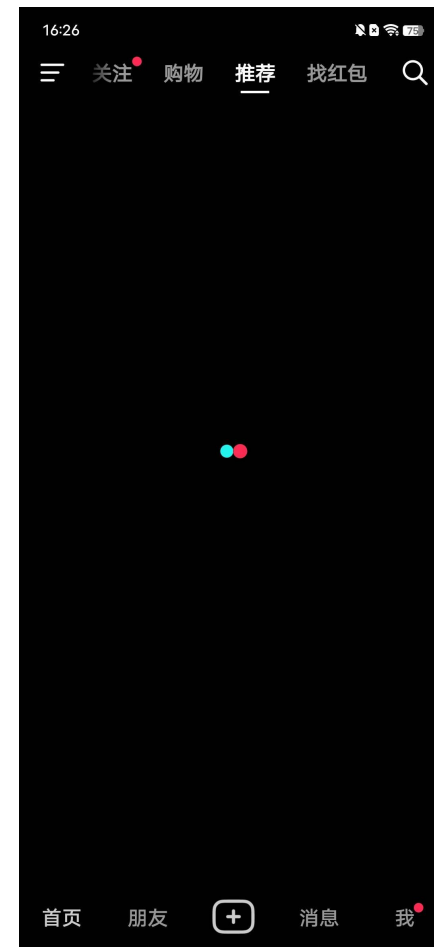
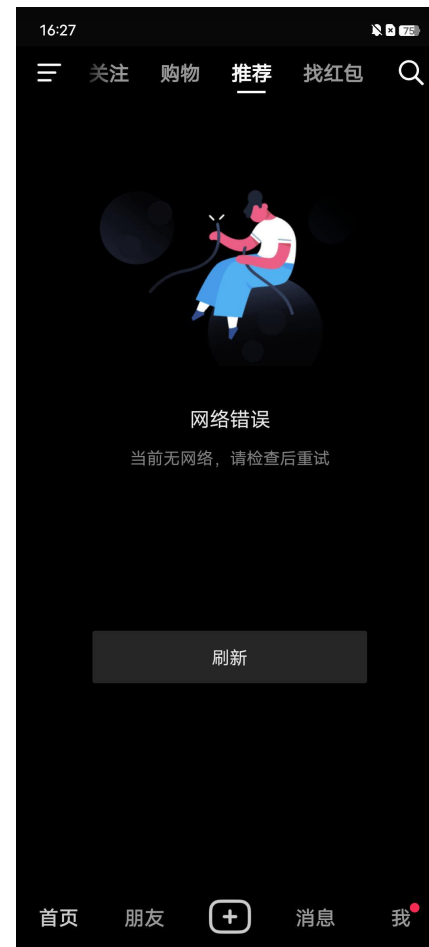
01

背景介绍

01_为什么要优化网络?

直接关系到App的可用性及基本的用户体验。

- Google研究表明, 页面加载时间超过3s, 53%的用户会选择离开
- Amazon页面加载延长1秒, 一年就会减少16亿美金营收
- BBC发现网页加载每增加一秒钟, 他们就会失去总用户的10%。



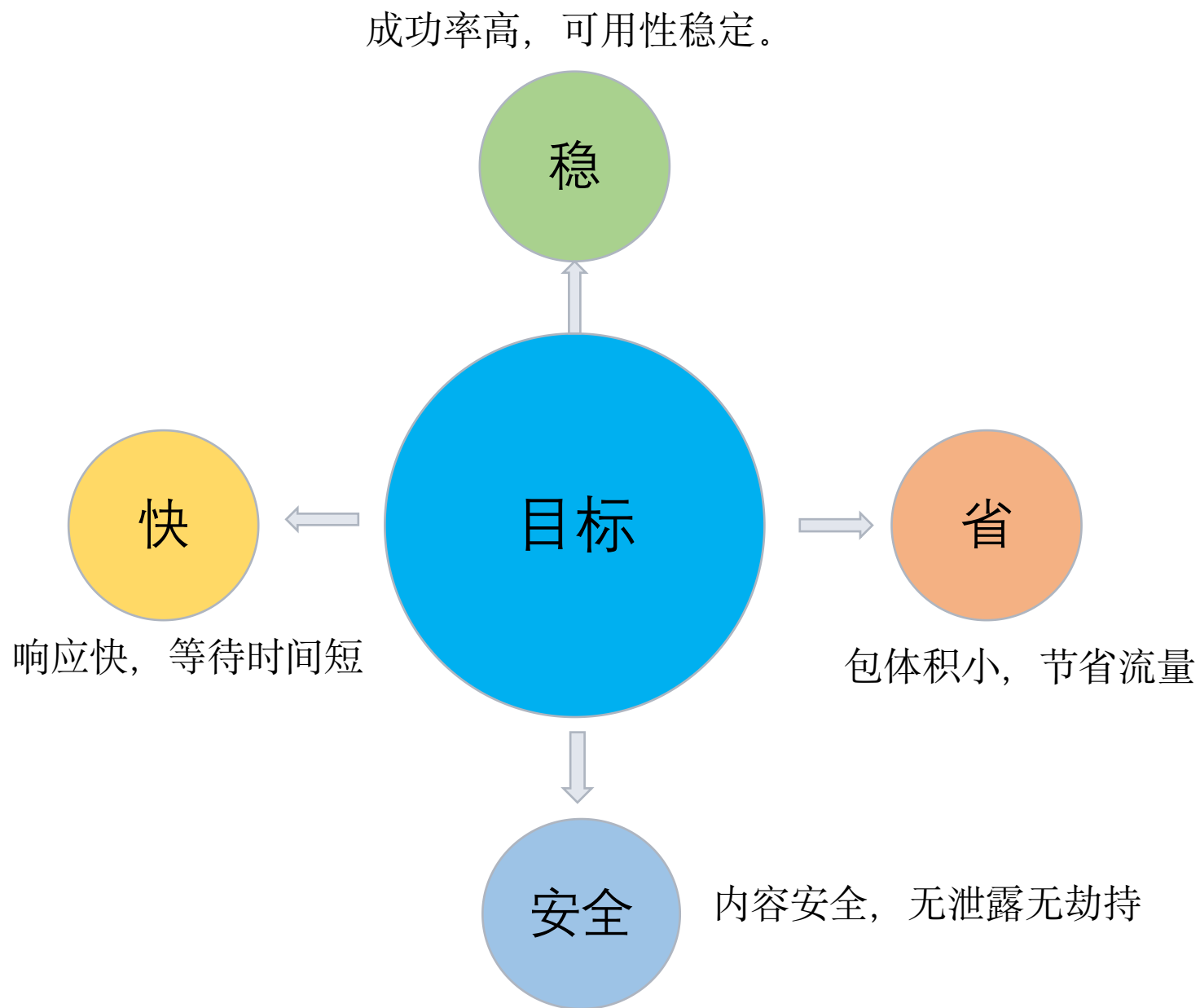
01_为什么要优化网络?

网络成本是公司研发的重要组成部分

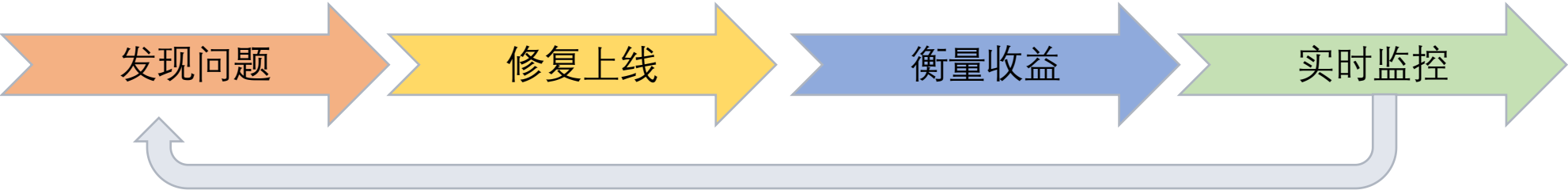
- 国内带宽成本按照**95**峰值计费
- 1Tbps API带宽成本 (DSA + TLB) 近1.8亿元/年



01_优化目标



01_优化流程



发现问题	监控系统、用户报障、线下测试
修复上线	问题归因、问题修复、AB实验
衡量收益	性能收益、业务收益、理论闭环
实时监控	实时监控、及时报警

02

网络监控

02_网络监控

工欲善其事，必先利其器

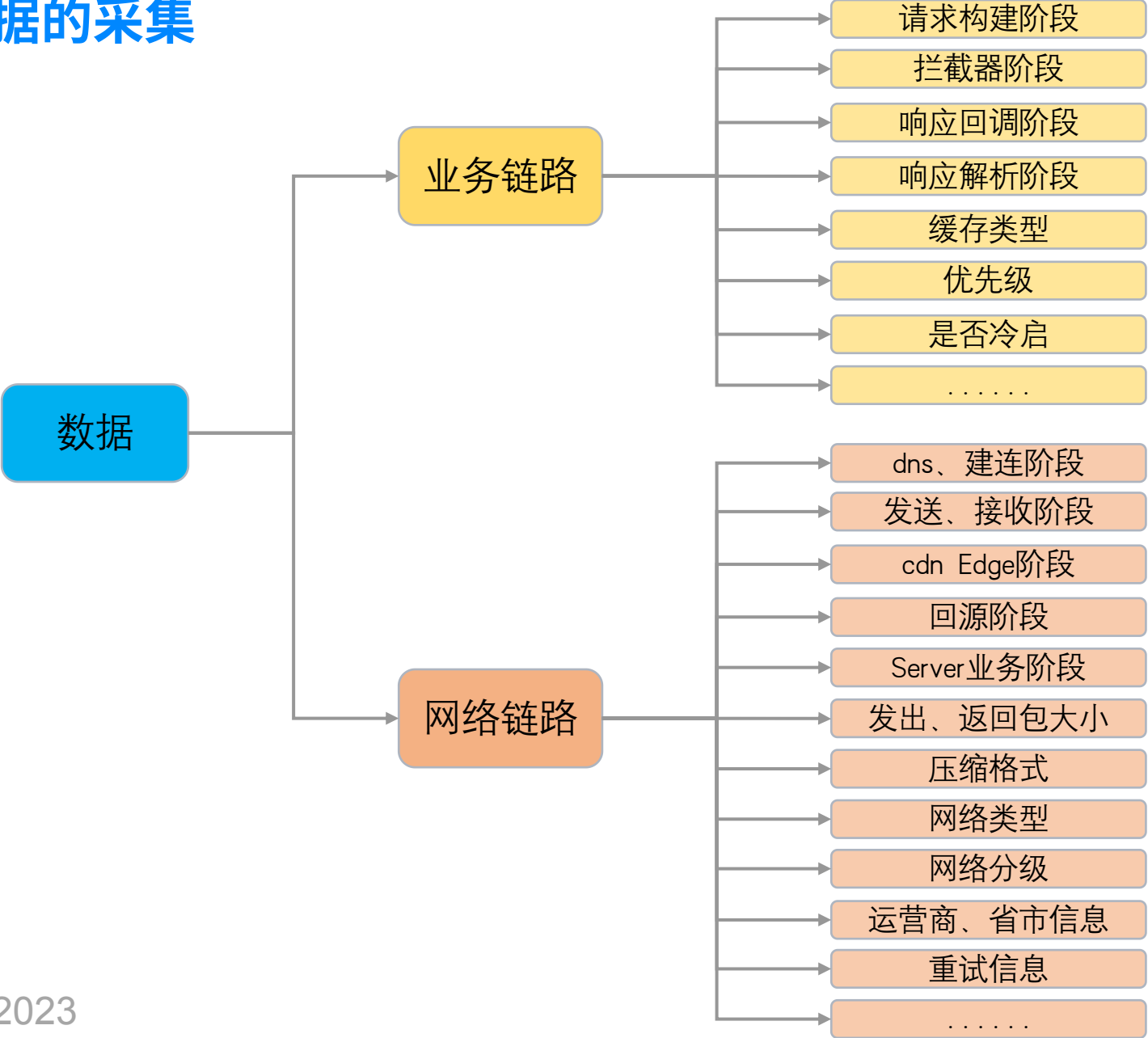
全：数据完备，尽量包更多维度，覆盖更多的环节

准：数据准确置信，指标稳定不波动

快：报警及时，定位问题快



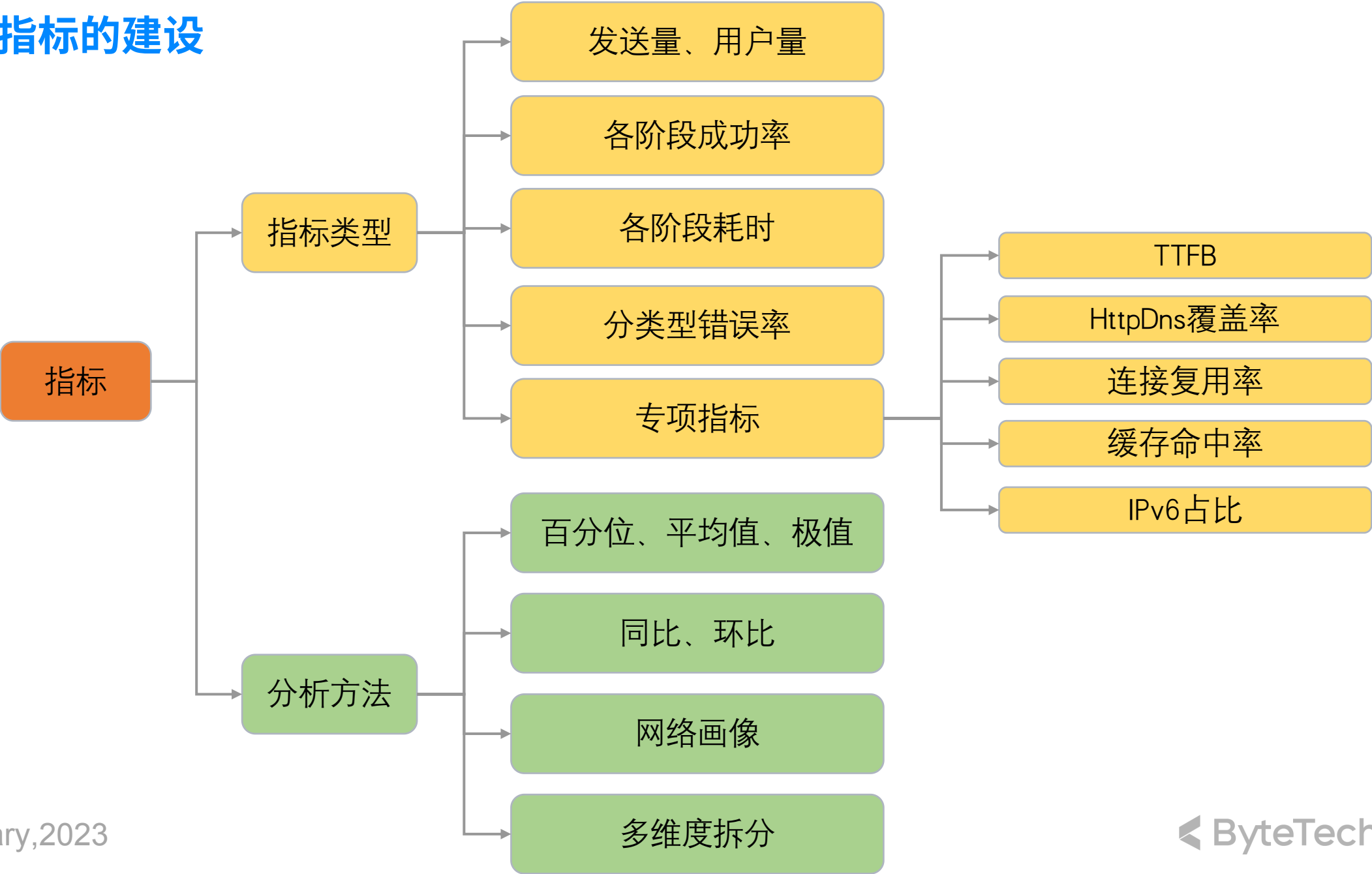
02_数据的采集



基准采样率:
保证大盘的稳定性

业务自定义采样率:
保证较小业务的指标稳定性

02_指标的建设



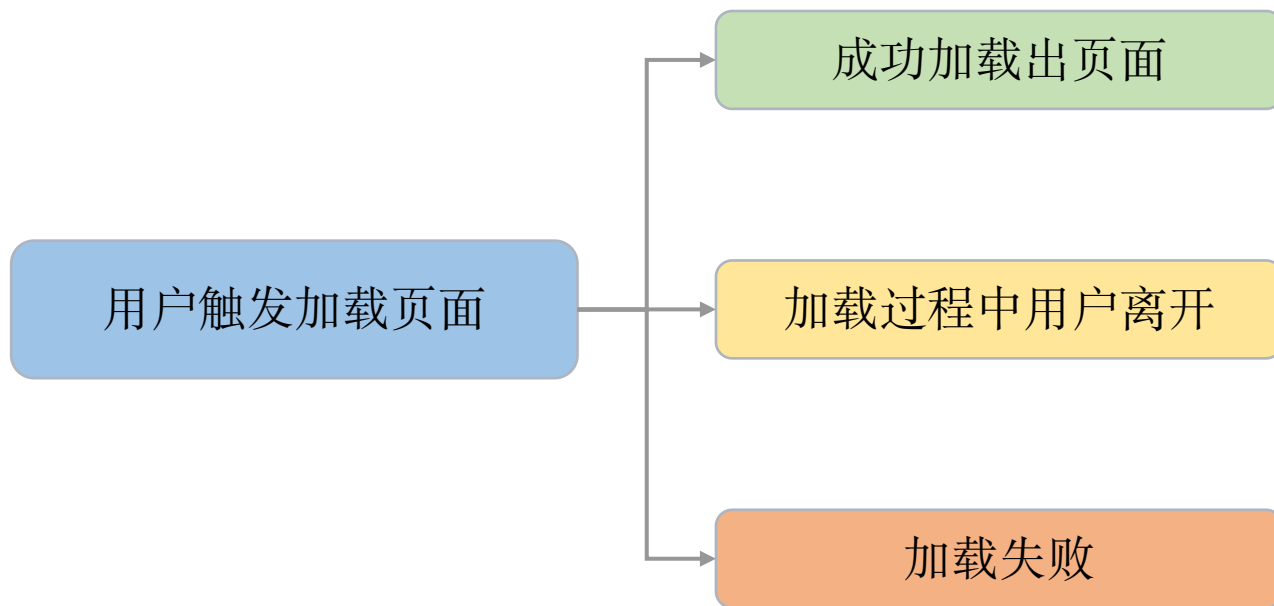
03

体验优化

01_优化思路

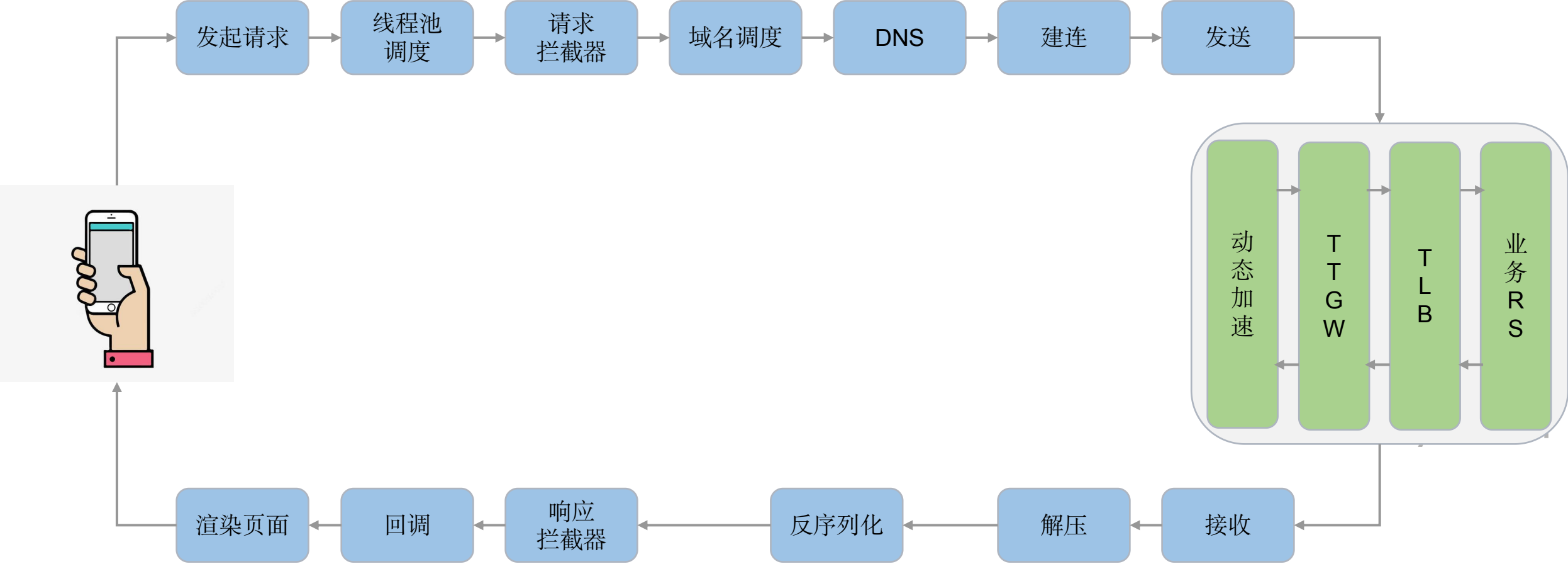
从用户视角出发，优化最真实的用户感受

- 客户端 + 服务端
- 网络链路 + 业务链路



03_优化思路

分析请求链路各个环节，针对性地制定策略



03_案例1：拦截器的优化

拦截器是网络库中最常见的设计模式，但是也容易造成问题

```
}  
override fun intercept(chain: Interceptor.Chain): SsResponse<*> { chain: RealInterceptorChain@76705  
    ALog.i(TAG, "enter probe interceptor")  
    var response = chain?.proceed(chain?.request())  
    var headers= response.headers() as List<Header>  
    var value : String? = null  
    ALog.i(TAG, "header: $headers")  
    ...  
    map[ACTIVITY_LITE_CONF_SETTINGS_VERSION]?.let { stringVersion ->  
        curVersion = stringVersion.toIntOrNull() ?: -1  
        val preVersion = PolarisKevaHelper.getIntValue(ACTIVITY_LITE_CONF_SETTINGS_VERSION, -1)  
        ALog.i(TAG, "current version is $curVersion,preversion is $preVersion")  
        if(preVersion == -1){  
            saveCurVersion()  
        }  
        else if(preVersion != -1 && curVersion != -1 && preVersion < curVersion){  
            ALog.i(TAG, "try fetch, current version is $curVersion,preversion is $preVersion")  
            ServiceManager.get().getService(IActivitySettingsManager::class.java).fetch("feed", nu  
        }  
    }  
    return chain.proceed(chain.request())  
}
```

业务开发不熟悉网络代码，导致事故

```
java.lang.StackOverflowError  
  
    at java.nio.MemoryBlock.peekInt(MemoryBlock.java:224)  
    at java.nio.DirectByteBuffer.getInt(DirectByteBuffer.java:218)  
    at com.android.dex.Dex.annotationDirectoryOffsetFromClassDefIndex(Dex.java:482)  
    at java.lang.Class.getDexAnnotationDirectoryOffset(Class.java:1313)  
    at libcore.reflect.AnnotationAccess.getAnnotationSetOffset(AnnotationAccess.java:216)  
    at libcore.reflect.AnnotationAccess.getAnnotationReader(AnnotationAccess.java:558)  
    at libcore.reflect.AnnotationAccess.getOnlyAnnotationValue(AnnotationAccess.java:600)  
    at libcore.reflect.AnnotationAccess.getSignature(AnnotationAccess.java:448)  
    at java.lang.Class.getTypeParameters(Class.java:1070)  
    at com.google.gson.internal.$Gson$Types.resolveTypeVariable(.java:429)  
    at com.google.gson.internal.$Gson$Types.com_google_gson_internal_$Gson$Types__resolve$__twin__  
    at com.google.gson.internal.$Gson$Types.com_google_gson_internal_$Gson$Types_com_ss_android_ugc_  
    at com.google.gson.internal.$Gson$Types.resolve(.java:337)  
    at com.google.gson.internal.$Gson$Types.com_google_gson_internal_$Gson$Types__resolve$__twin__  
    at com.google.gson.internal.$Gson$Types.com_google_gson_internal_$Gson$Types_com_ss_android_ugc_  
    at com.google.gson.internal.$Gson$Types.resolve(.java:337)  
    at com.google.gson.internal.$Gson$Types.resolve(.java:337)  
    at com.google.gson.internal.$Gson$Types.resolve(.java:337)  
    at com.google.gson.internal.$Gson$Types.getSupertype(.java:283)
```

拦截器膨胀，层级太多影响性能，甚至导致
StackOverflowError

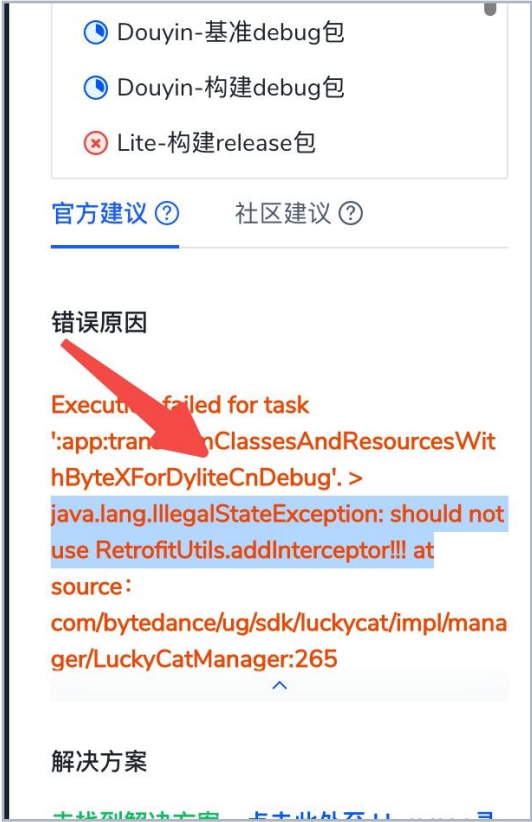
03_案例1：拦截器的优化

- "拍扁"拦截器，将多个拦截器合并成一个
- 只管具体的业务逻辑，拦截器的流转对其透明

NetworkPartner改造

角色名称	作用
<i>RequestHandler</i>	在发出网络请求前，对请求进行处理
<i>ResponseHandler</i>	在收到网络响应后，对响应进行处理
<i>ExceptionHandler</i>	对异常进行捕获和处理
<i>Blocker</i>	对请求进行拦截，返回特定响应
<i>Bystander</i>	网络过程的观察者

- 管控全局拦截器防劣化，编译期拦截MR



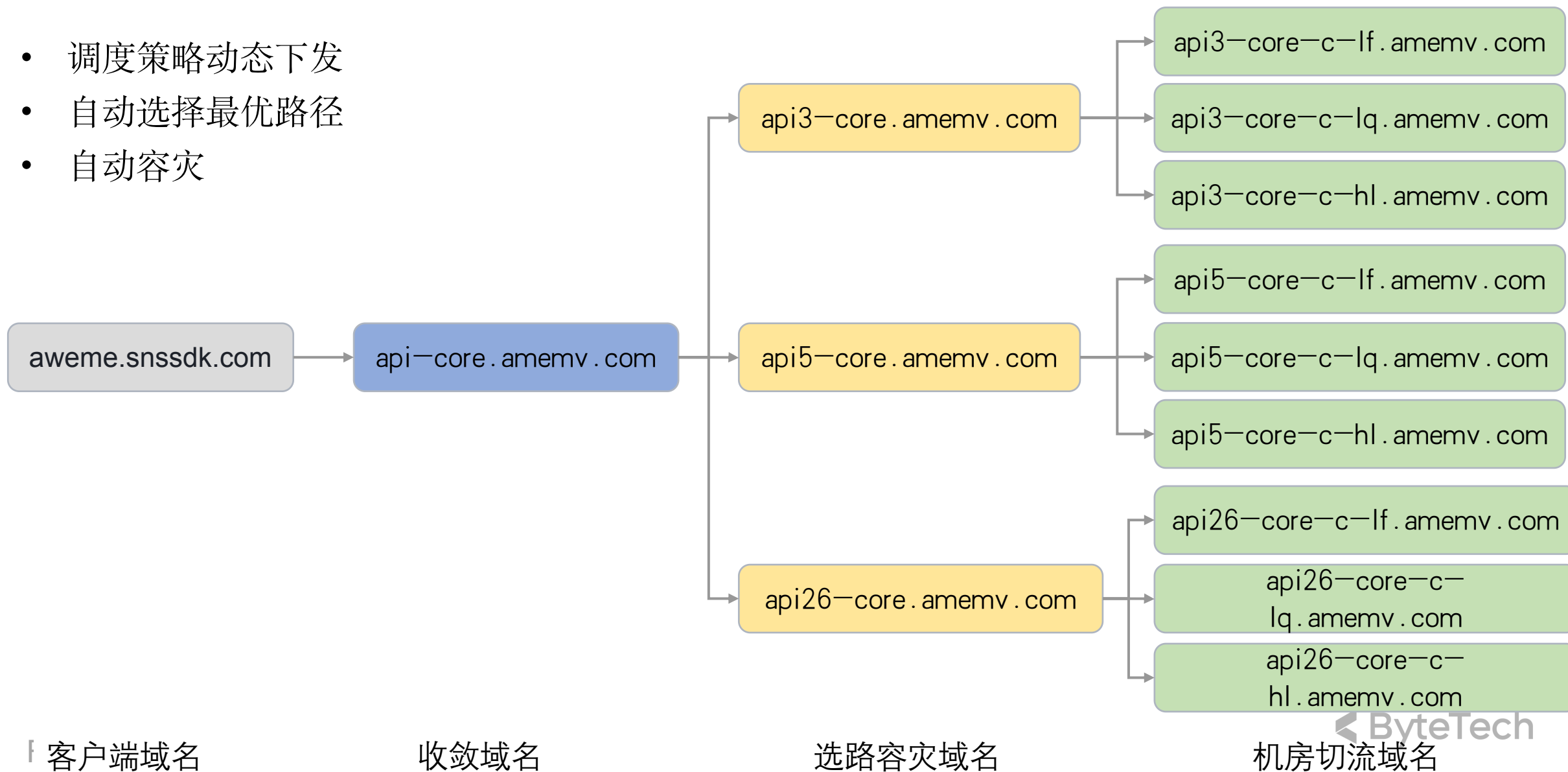
03_案例2：请求调度

- 为什么需要调度？

用户所在地	北京	天津	南京	上海	杭州	广州	深圳	
运营商	 中国移动 China Mobile	 China unicom中国联通			 中国电信 CHINA TELECOM			
DSA厂商	 阿里巴巴 Alibaba.com			 HUAWEI		 ByteDance		
核心机房	LF			HL		LQ		

03_案例2：请求调度

- 调度策略动态下发
- 自动选择最优路径
- 自动容灾



03_案例3: DNS的问题

默认采用运营商的LocalDns

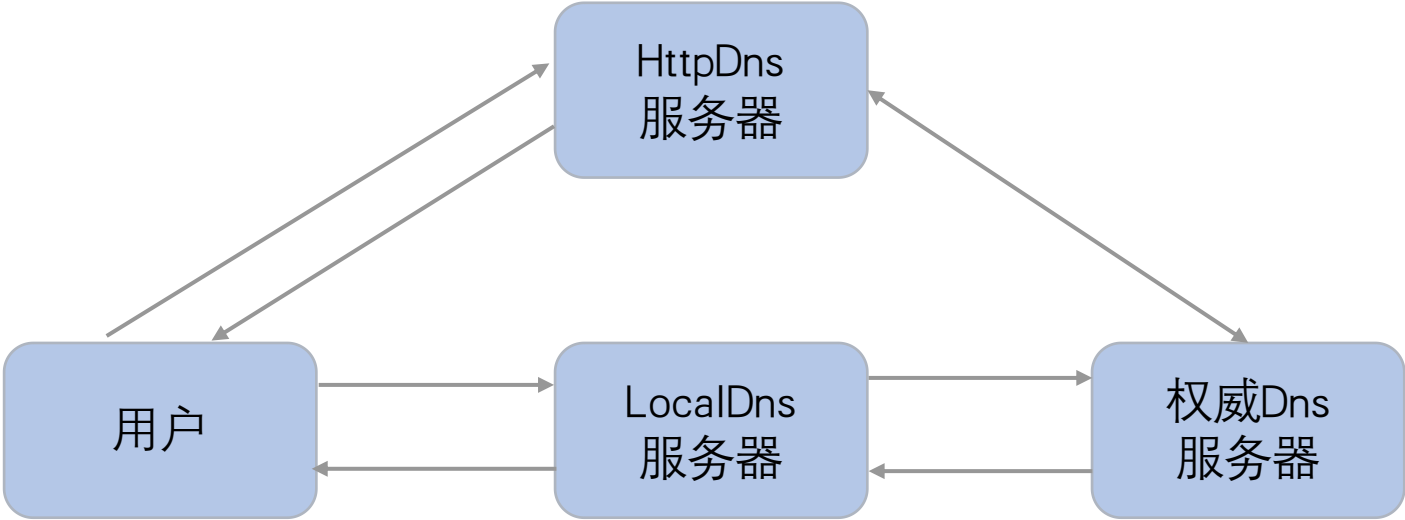
- 采用UDP协议, 通道不安全, 容易被劫持
- 缓存更新慢, 切流时间长
- 每次查询一个域名, 效率低

部分域名TTL设置不合理

- TTL设置过低



03_案例3：DNS的优化

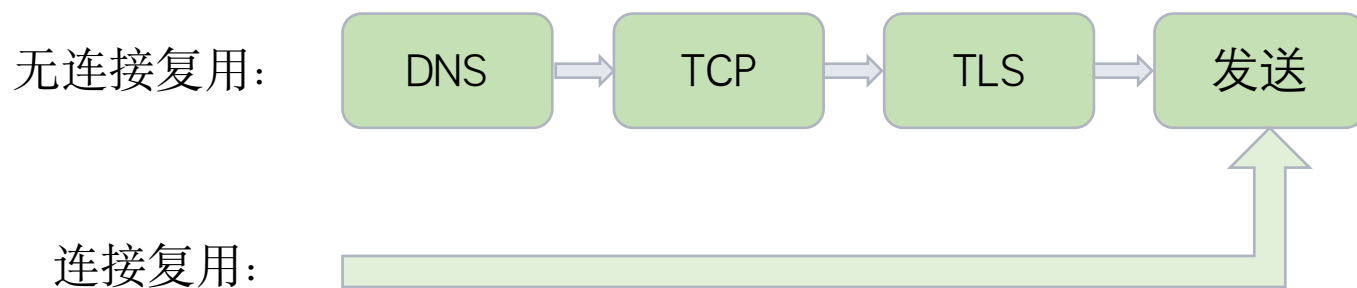


策略名称	作用
HttpDns	效率高，防劫持，智能调度，灾备
TTL调整	提高DNS缓存命中率
兜底IP	DNS失败后提供一个可用兜底IP
收益	DNS阶段成功率>99% 核心域名HttpDns覆盖率95.8%

03_案例4：连接复用率

客户端的连接建立过程是比较耗时的，以HTTPS为例：

- DNS: 1 RTT
- TCP: 1.5 RTT
- TLS: 2-3 RTT



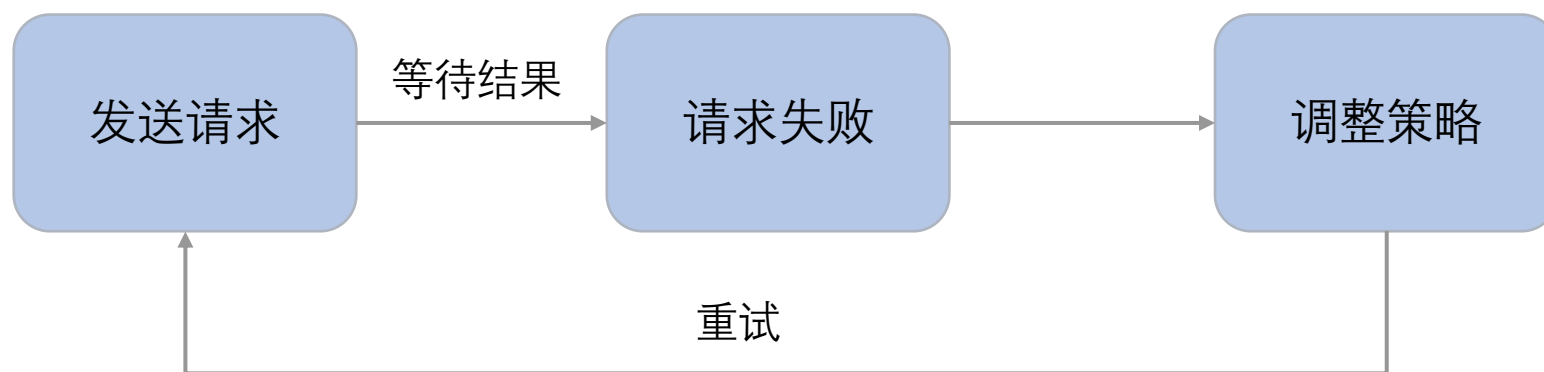
03_案例4：连接复用率

- 针对核心场景的域名，提升连接复用率，可以有效减少耗时

策略名称	解释	副作用
预建连接	在真实触发请求前，提前创建连接，放入连接池	需要关注策略带来的后端连接数的提升以及客户端的资源消耗
连接保活	在连接空闲时，通过极小的代价，保活连接	
连接重建	检查连接是否断开，如果断开重建连接	
收益	核心域名连接复用率 95.9% ，建连耗时P80 2ms	

03_案例5：复合请求

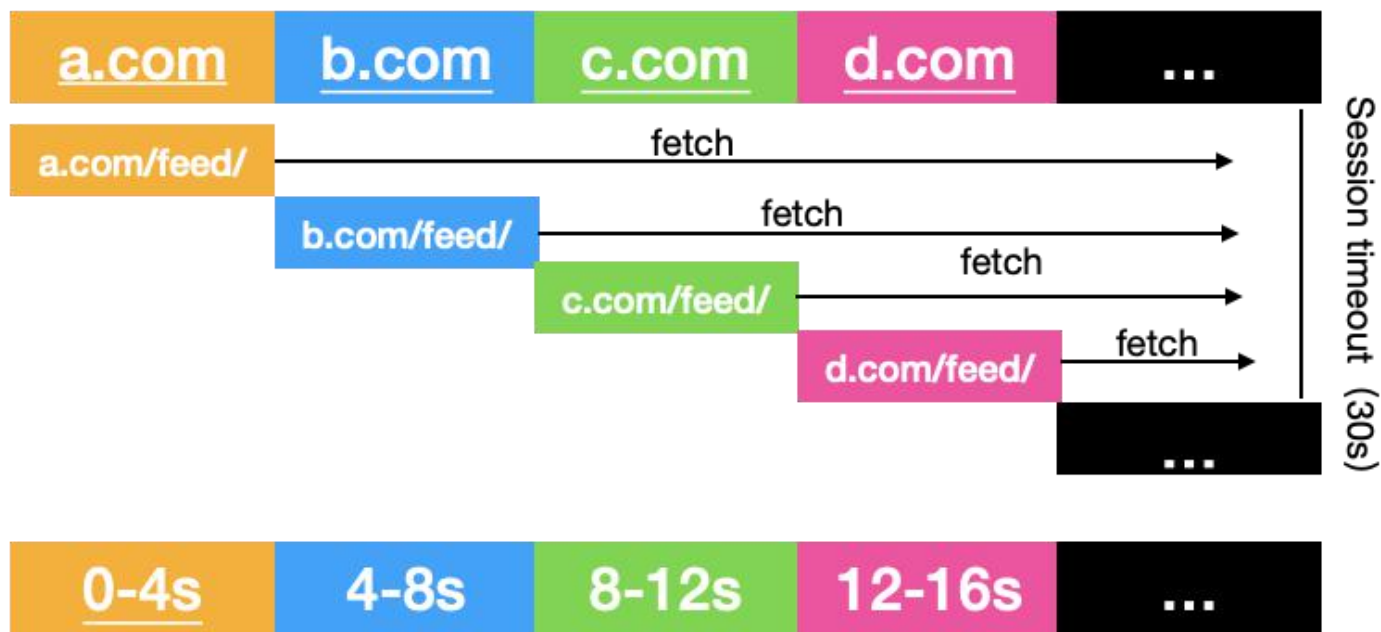
由于移动网络的波动性，对出错的请求进行重试能够有效提高成功率。而一般的请求重试方案，需要等待请求结果明确为失败了，才会调整发送的策略，再次发送请求进行重试。



03_案例5：复合请求

基于等价域名，开发复合请求方案，能够高效重试

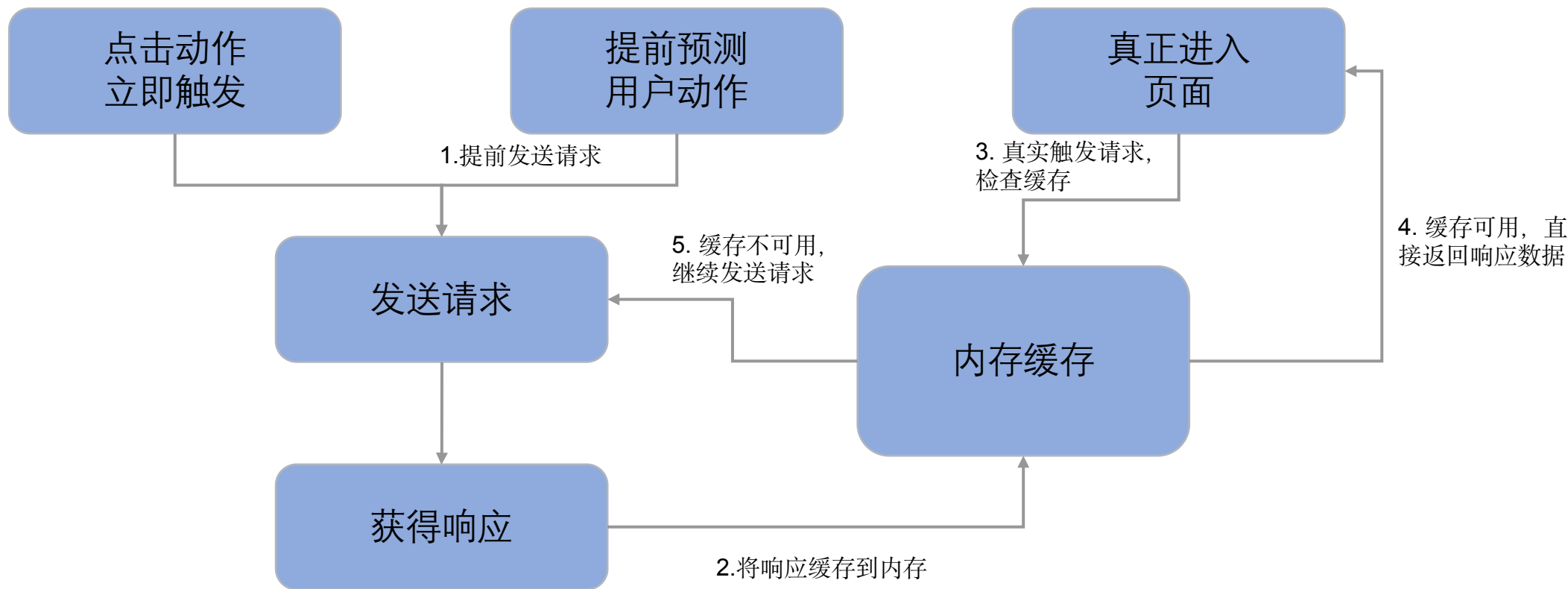
- 无需等待请求结果，一定时间内请求未结束，直接开始重试
- 可以并行请求多个域名
- 端上主动容灾



收益：抖音时刻接口成功率+1.8%

03_案例6：内存缓存

- 针对耗时敏感场景，提前触发请求，节省请求的网络耗时



收益：音乐详情页首次进入耗时-276ms，人均进入页面数+0.17%

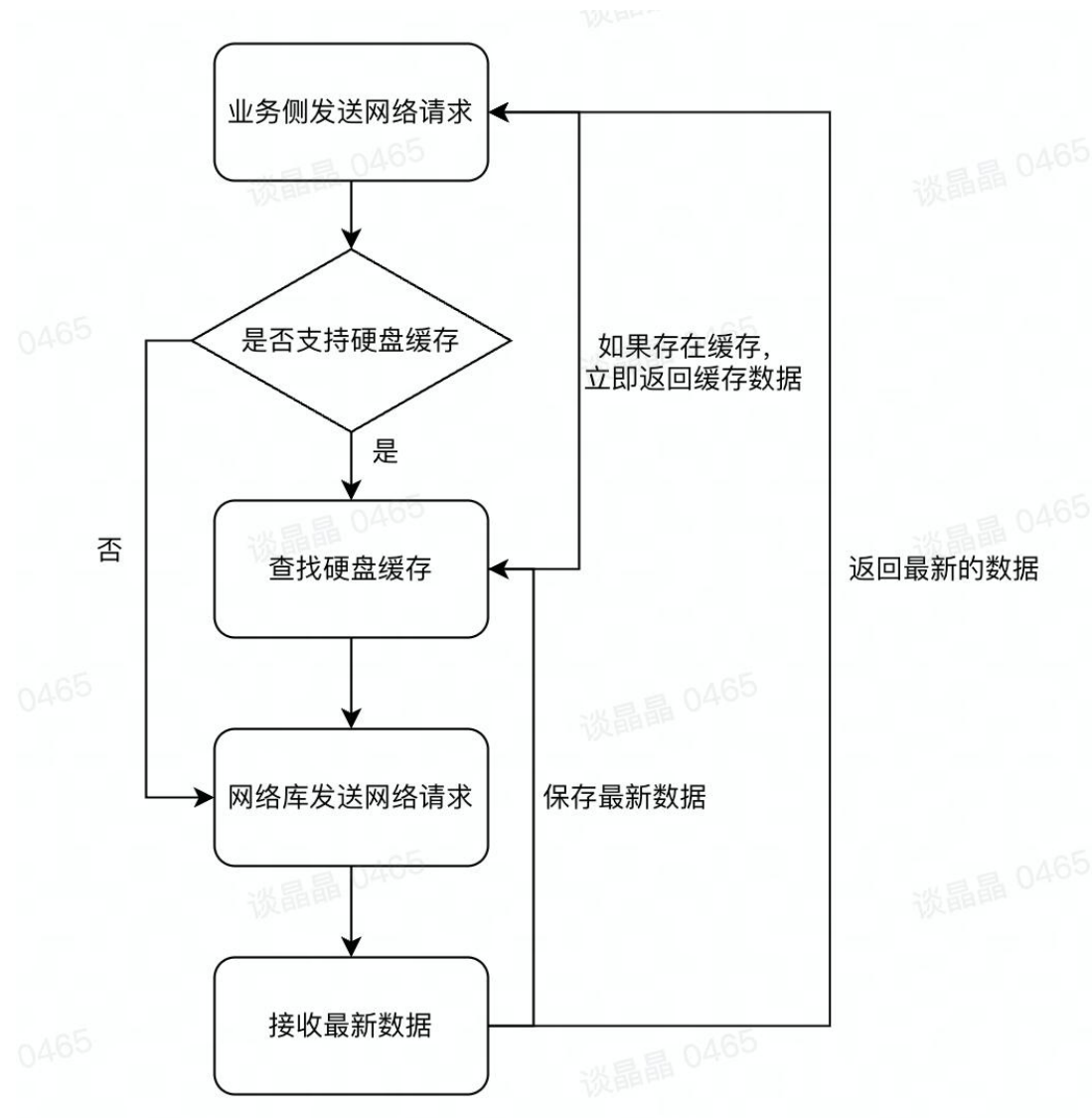
03_案例7：硬盘缓存

针对内容改变不频繁的场景

- 与其用户看loading页面/空白页面，不如让用户看旧数据。

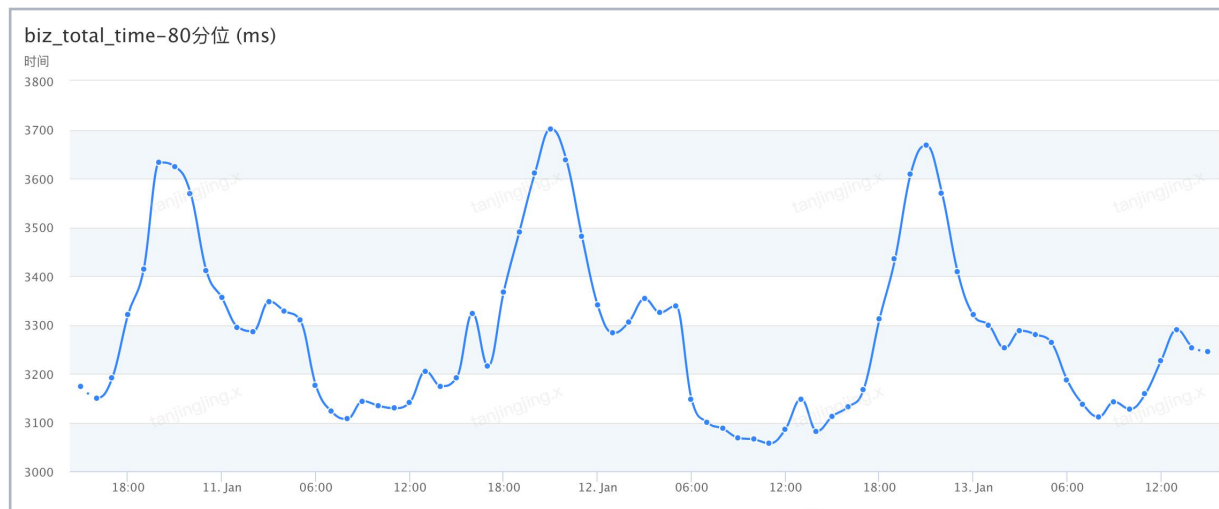
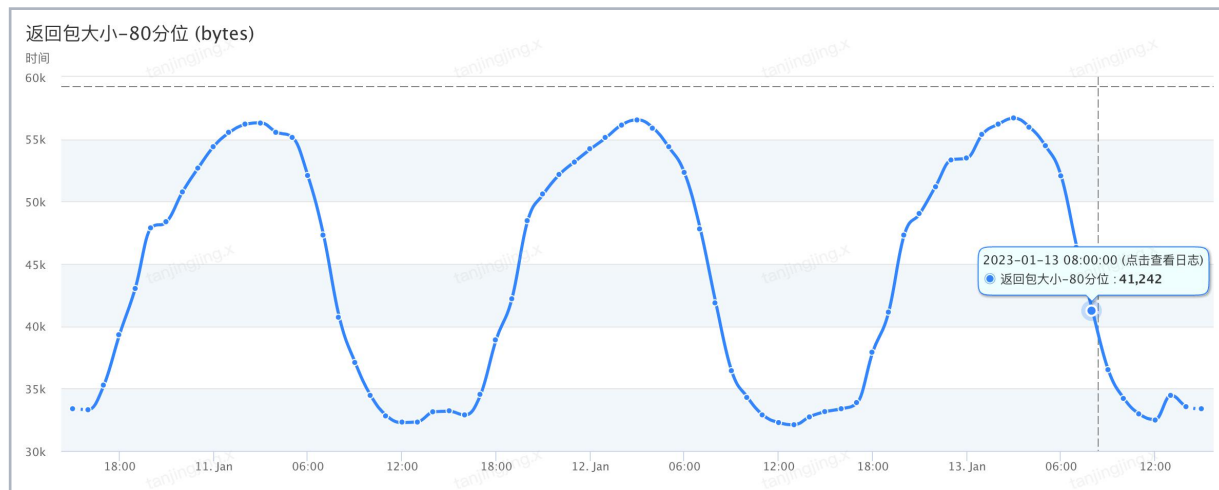
收益：

- 个人页
 - 收藏列表耗时-43%
 - 人均视频播放数+0.3%



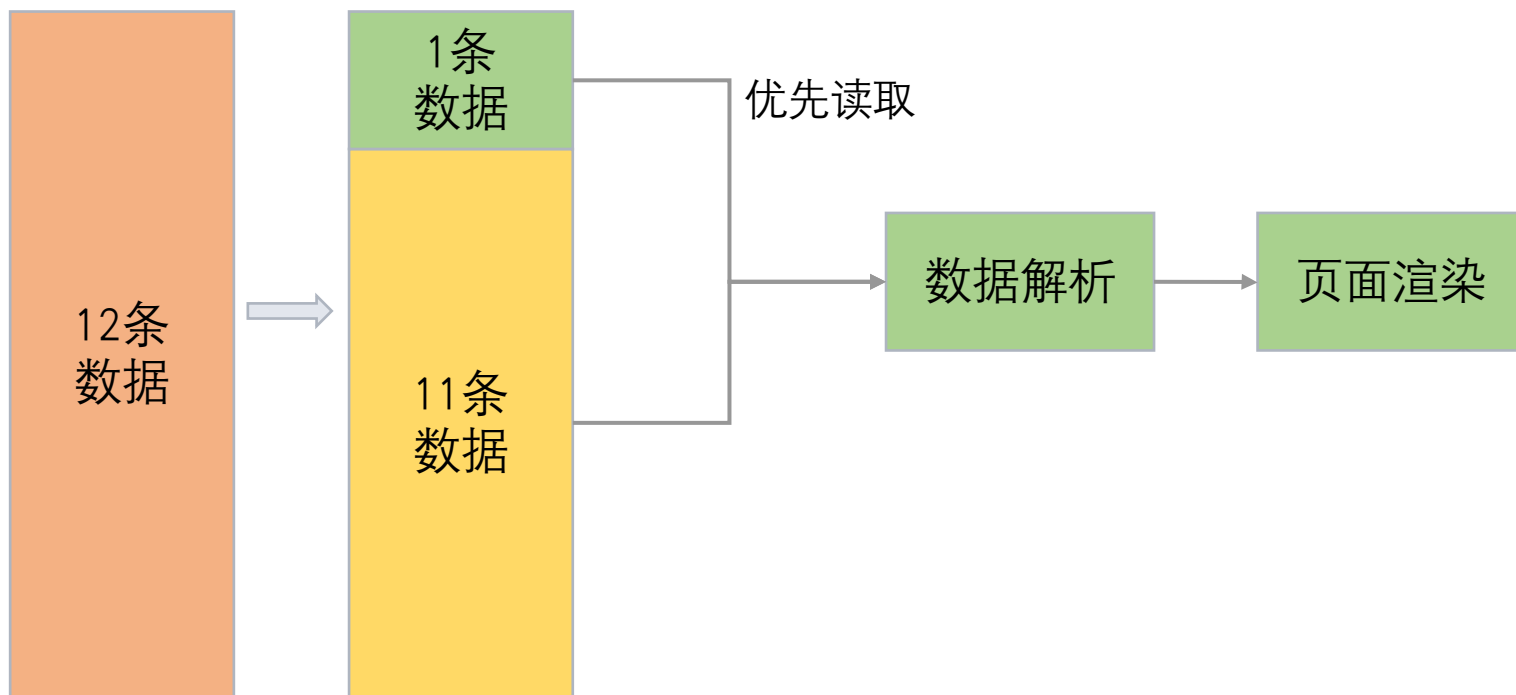
03_案例8：响应分段

- 请求的总耗时与响应的包大小密切相关



03_案例8：响应分段

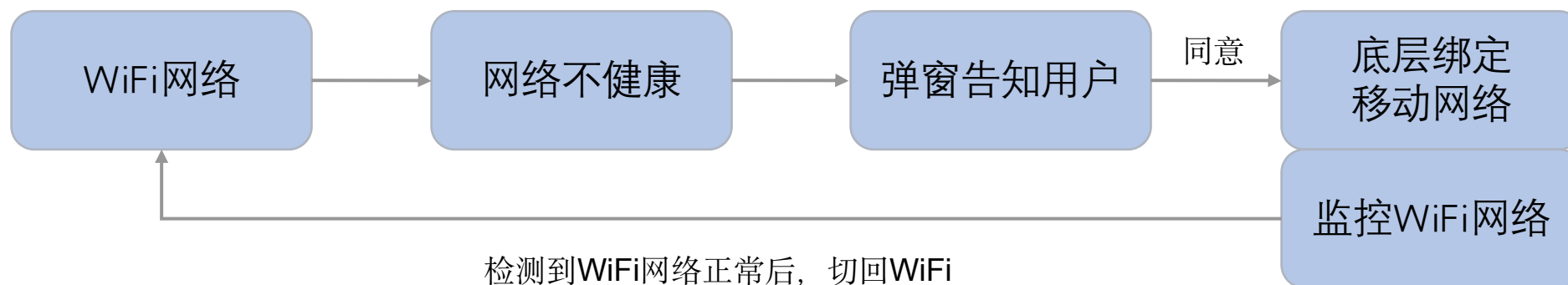
- 将大块的数据分段，优先读取和解析较小的数据段，减少交互阻塞时间



收益：首刷耗时-200ms

03_案例9: Wifi切移动网络

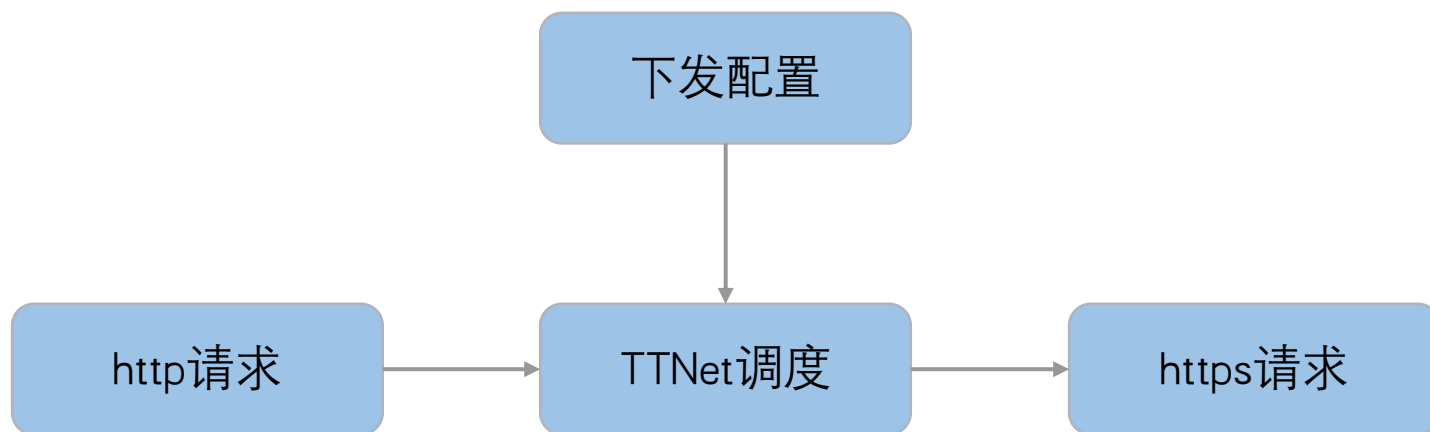
- 针对流量不敏感用户，在WiFi网络弱网/假网时，将网络绑定到移动网络，保证用户的体验



收益: 人均活跃天数+0.14%，人均使用时长+0.27%

03_案例10：网络安全优化

- http协议切换成https协议
 - ✓ 支持动态下发配置，根据域名和path切换
- 下掉低版本的TLS (tls1.0/tls1.1)

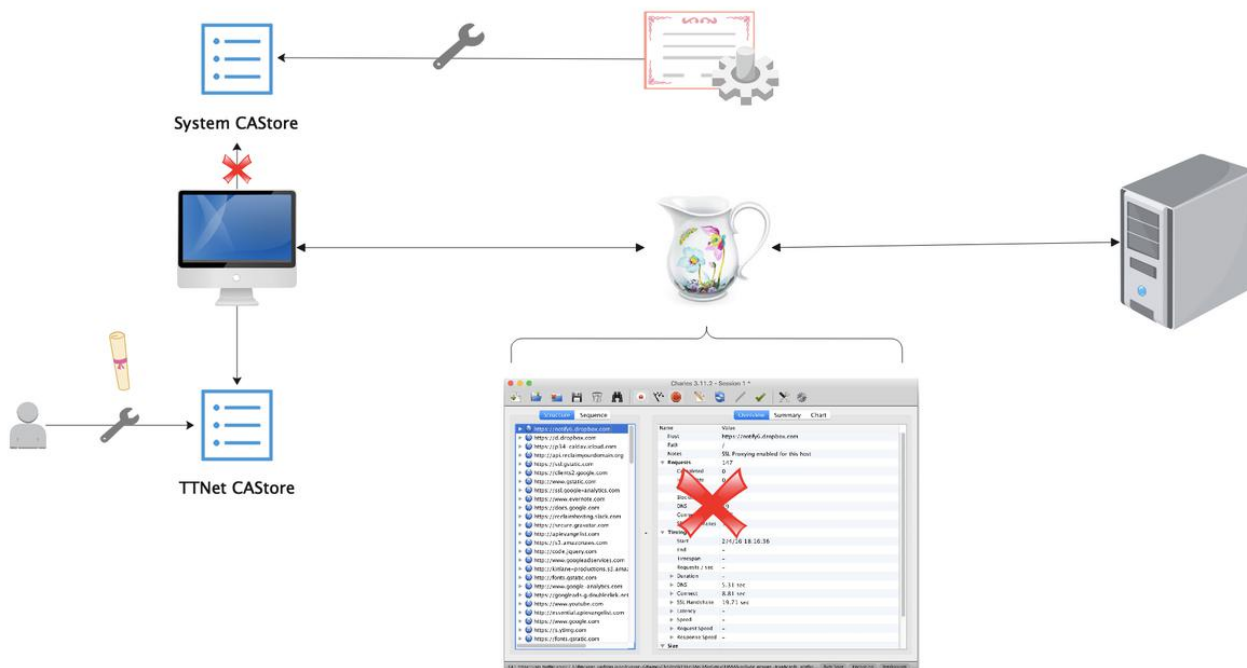


抖音https占比97%

03_案例10：网络安全优化

自研CA Store方案

- 防止Charles等工具抓包，泄露内容
- 初始化的时候注入CA证书集，在进行SSL证书认证的过程中，不再使用系统安装的根证书集合，而是使用注入的证书集合进行认证。



04

带宽优化

04_优化思路



删：删除无用



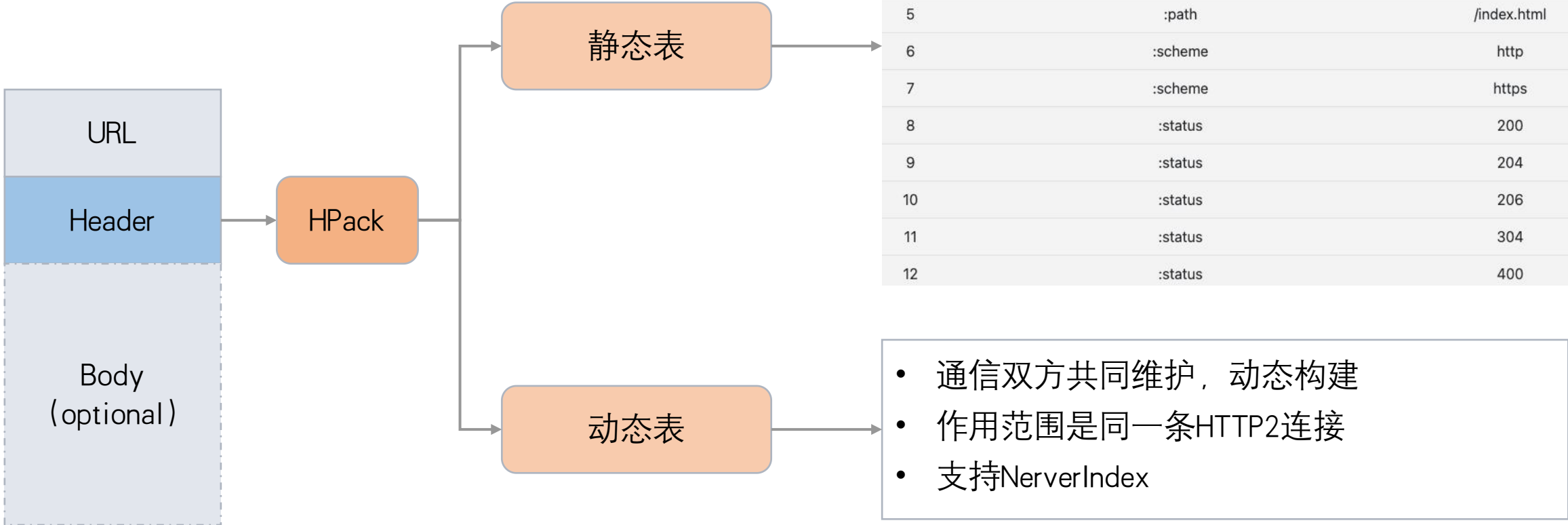
压：提高压缩比



削：削减峰值

04_上行带宽优化

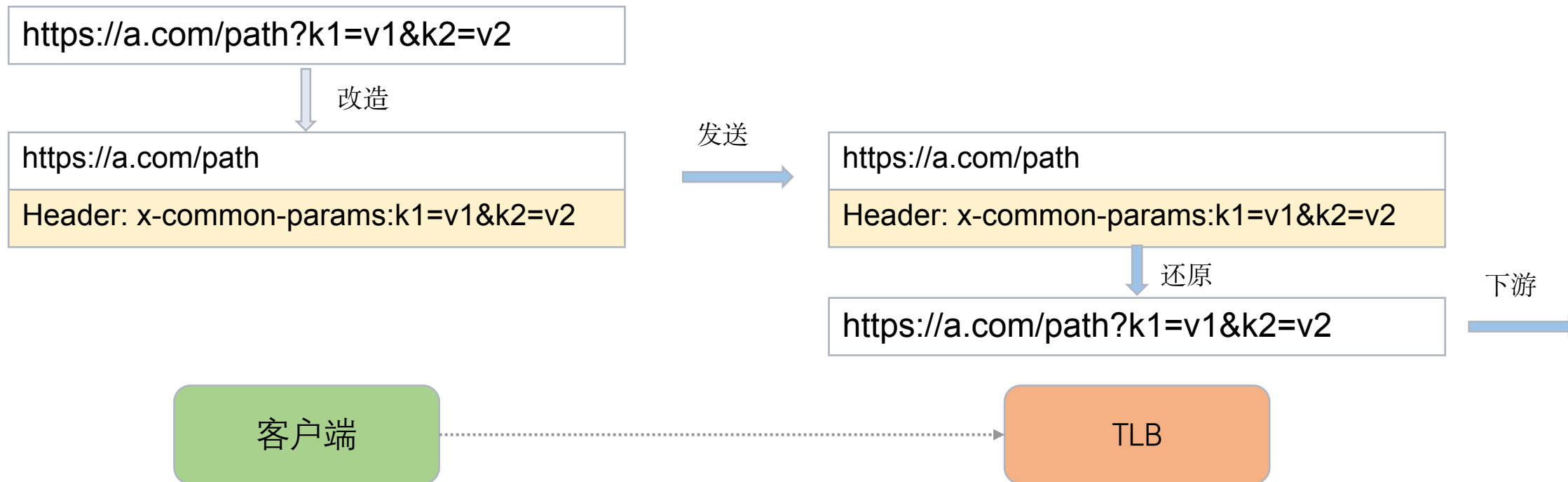
- H2 HPack原理：将重复的Header用索引代替



04_上行带宽优化

HPack极限优化

- 通常query参数中包含大量的通用参数
- 客户端将固定不变的部分拆分出来，放入Header，使用Hpack压缩



收益: Feed接口请求平均包大小-19%

04_下行带宽优化

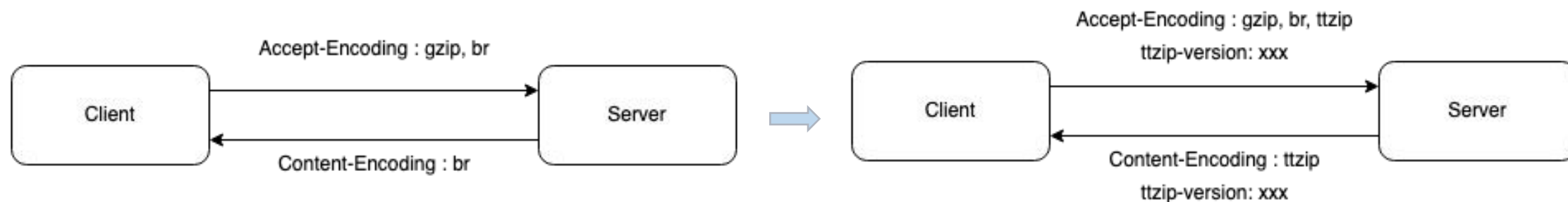
- 提高压缩比，提升压缩/解压效率
- zstd压缩算法
 - ✓ 支持训练字典，使用字典压缩/解压缩
 - ✓ 做到了压缩比和压缩性能兼得



04_下行带宽优化

zstd压缩算法的落地

- 支持字典的自动训练、测试、防劣化
- 新增一个ttzip-version来控制是否使用字典



05

长期规划

05_长期规划

网络监控

- 智能归因平台

体验优化

- 与业务场景紧密结合
- 环境、设备针对性优化
- 协议层精细化调优

带宽优化

- 业务裁剪
- 增量更新

05_致谢

- 本文中涉及的优化策略很多是基础体验团队、网络中台团队及抖音基础技术团队合作完成，在此对团队所有成员表示衷心感谢!
- 特别感谢李孟杰对本文内容的大力支持!



THANKS.

February, 2023